

Módulo 1 · Diseño de Sistema

Arquitectura, patrones, modelo de datos y diagrama de componentes. Autor: Diego Alejandro Sepulveda Huetio.

1. Cómo funciona la plataforma (visión de negocio)

La **Plataforma de Gestión de Pólizas de Arrendamiento** administra pólizas **individuales** y **colectivas** (cánones de arrendamiento y riesgos amparados), con todo su ciclo de vida: emisión, agregado de riesgos, renovación con ajuste por IPC y cancelación. La operación crítica es la **sincronización con el CORE transaccional legado (Oracle WebLogic)**, que no debe bloquear ni degradar la experiencia del usuario final.

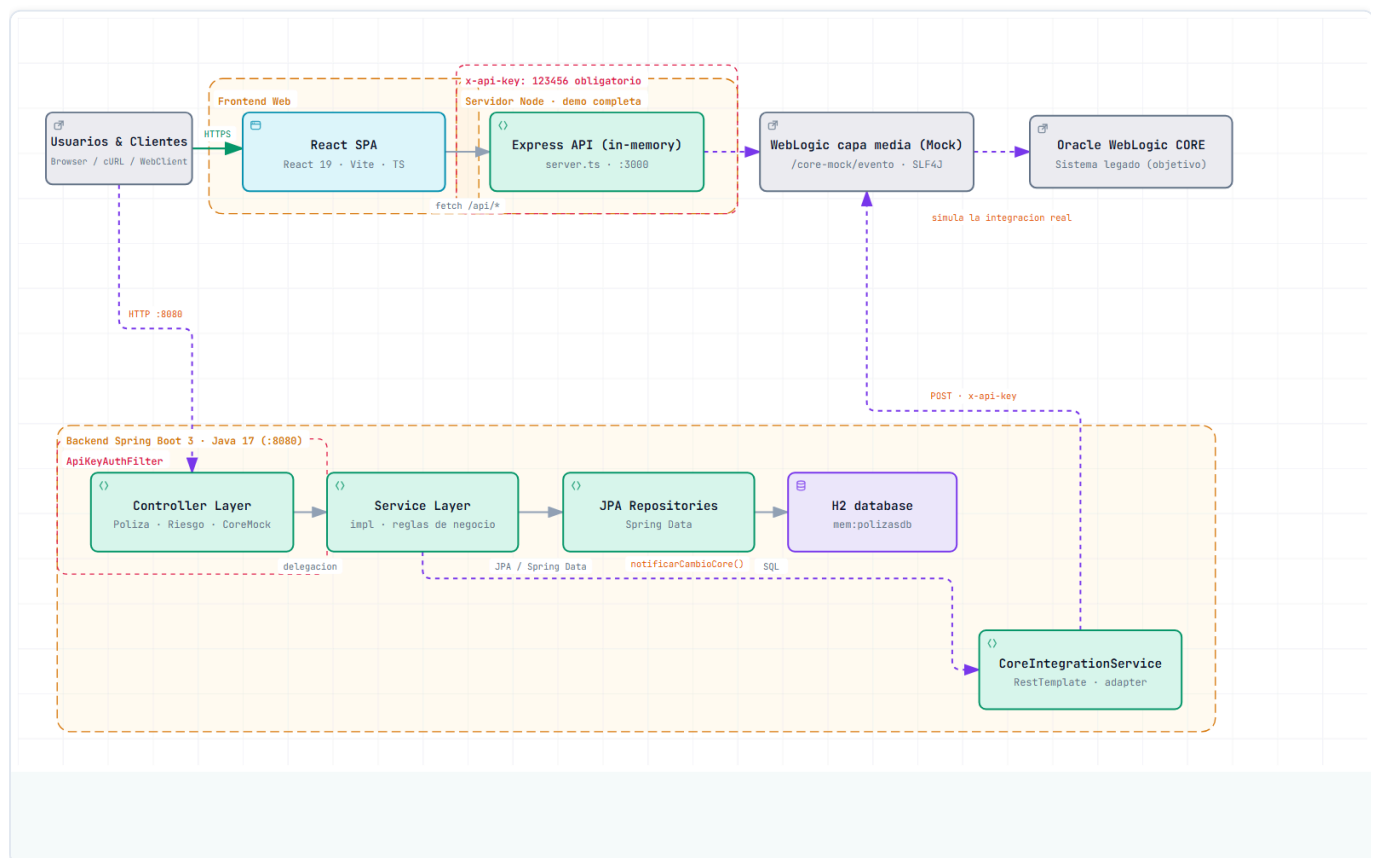


Figura 1. Arquitectura completa implementada y visualizada con Archify: frontend React 19, API demo Express (in-memory) y backend real Spring Boot 3 (Controller → Service → Repository → H2) con integración hacia WebLogic CORE.

2. Arquitectura de alto nivel y diagrama de componentes

- **API Gateway (Spring Cloud Gateway / Kong):** terminación SSL, validación de API Key/JWT, rate limiting y enrutamiento hacia los microservicios.
- **Servicio de Pólizas (Spring Boot):** ciclo de vida contractual — creación, cálculo de prima (`canon × vigencia_meses`), renovación periódica ajustada por IPC y cancelaciones.
- **Servicio de Riesgos (Spring Boot):** cardinalidad de inmuebles asegurados (1 para individual; 1..N para colectivas) y validaciones contractuales.
- **Bus de eventos (Apache Kafka / RabbitMQ):** desacopla operaciones críticas; cada cambio de estado emite `PolizaModificadaEvent` / `RiesgoCanceladoEvent`.

- **Adapter de Integración con CORE (WebLogic Connector):** consume eventos del broker y sincroniza el legado vía servicio agnóstico SOAP/REST (Outbox + Circuit Breaker).
- **Servicio de Notificaciones (Email/SMS Worker):** consume eventos de renovación/emisión y despacha correos (SendGrid) y SMS (Twilio/Infobip).
- **Base de datos:** PostgreSQL con réplicas de lectura y particionamiento por estado/fecha de vigencia.

En la implementación de la prueba, el backend real (`PolizaServiceImpl` , `RiesgoServiceImpl`) reproduce este diseño en micro: capas `controller` → `service` → `repository` , entidades `Poliza` y `Riesgo` , y el `CoreIntegrationService` hace el papel de adapter hacia la capa media (endpoint mock `/core-mock/evento` que registra logs SLF4J de cada intento de sincronización).

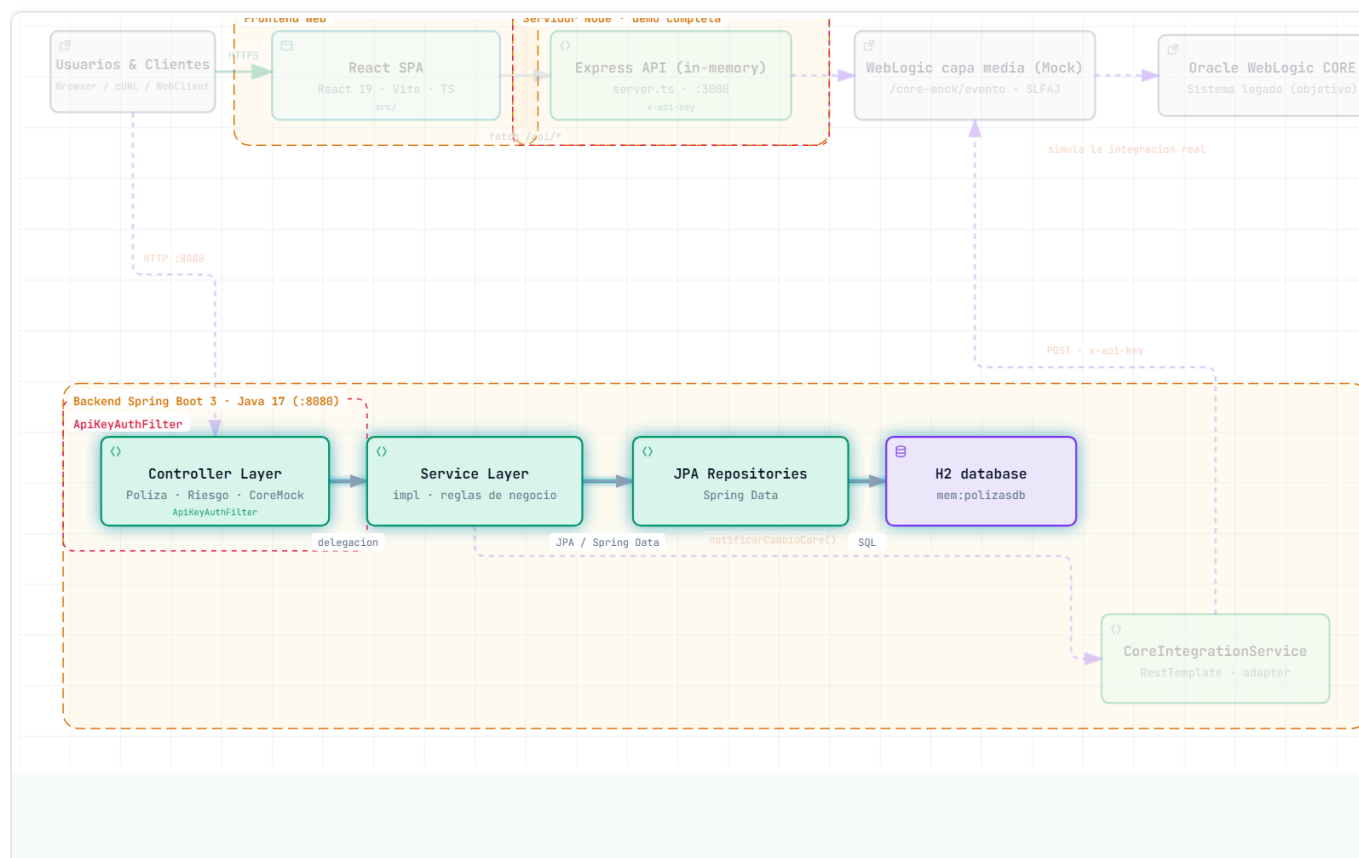


Figura 2. Vista "Backend real Spring Boot" (Archify): Controller → Service → Repository → H2 (Java 17, :8080), con filtro de seguridad `ApiKeyAuthFilter`.

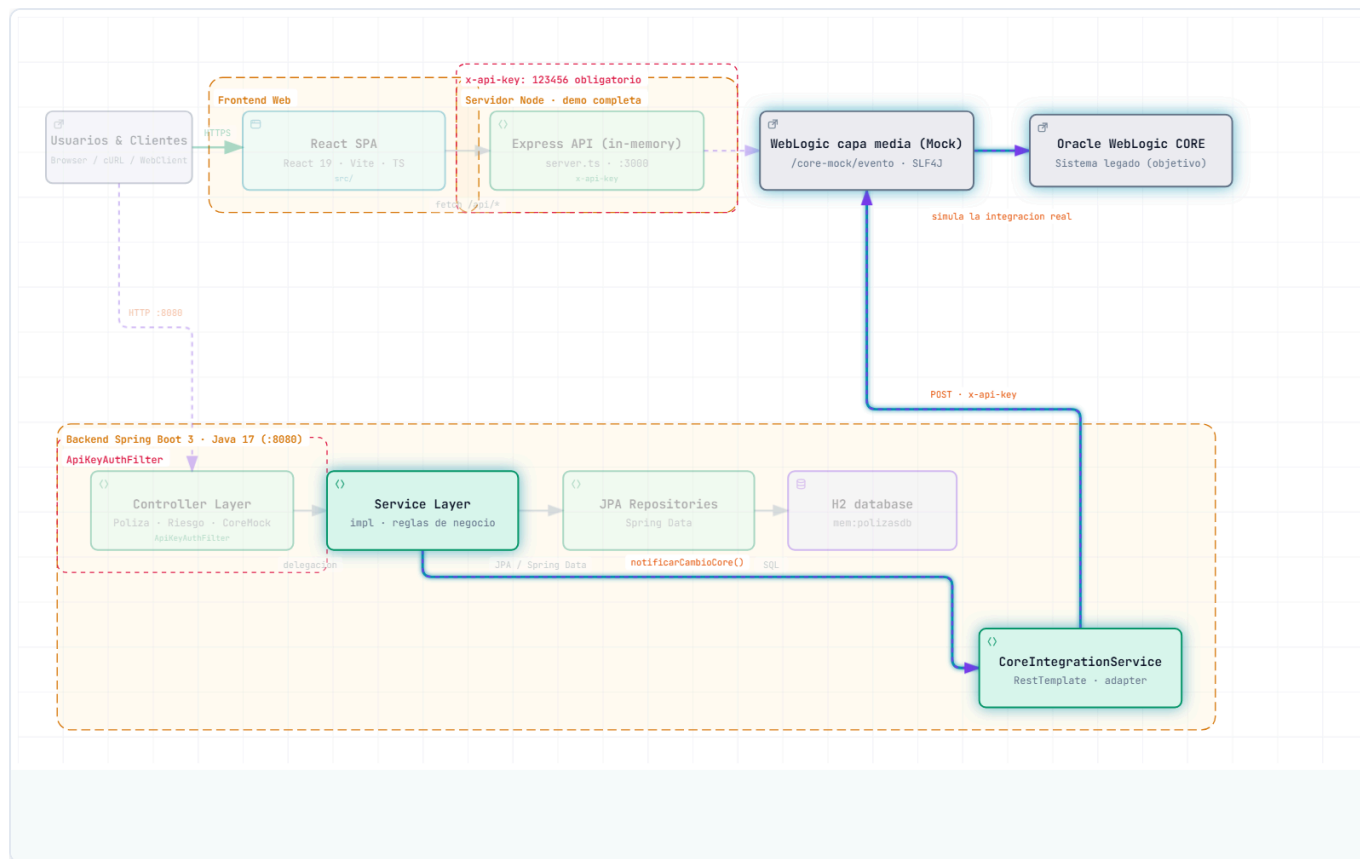


Figura 3. Vista "WebLogic CORE (Módulo 1)" (Archify): CoreIntegrationService (adapter) notifica a la capa media WebLogic (mock), el destino del patrón Hexagonal/Outbox.

3. Tres patrones de arquitectura justificados

3.1 Event-Driven Architecture (EDA)

- **Por qué se eligió:** la integración con el CORE legado y las notificaciones multicanal pueden ser lentas o caer; no deben afectar el front-end.
- **Cómo aplica aquí:** cada mutación de estado (CREACION_POLIZA , RENOVACION_IPC , CANCELACION_POLIZA_Y_RIESGOS , AGREGAR_RIESGO , CANCELAR_RIESGO) genera un evento que el bus despacha a consumidores.
- **Beneficio:** eventual consistency, resiliencia y confirmación inmediata al cliente.

3.2 Arquitectura Hexagonal (Ports & Adapters)

- **Por qué se eligió:** las reglas de negocio aseguradoras (fórmula de prima, límite 1 riesgo en individual, ajuste IPC) deben ser inmunes al framework, la BD y el legado.
- **Cómo aplica aquí:** las interfaces PolizaService / RiesgoService son los puertos; CoreIntegrationService es un adaptador saliente hacia WebLogic.
- **Beneficio:** testabilidad al 100 % y portabilidad (cambiar H2 → PostgreSQL u Oracle, o SOAP → REST, sin tocar el dominio).

3.3 CQRS + API Gateway

- **Por qué se eligió:** el tablero mensual y las consultas masivas tienen patrones de acceso distintos a las transacciones de renovación/emisión.

- **Cómo aplica aquí:** separa comandos (renovar, cancelar, agregar riesgo) de consultas optimizadas para reportes.
- **Beneficio:** modelos de lectura (vistas materializadas + Redis) que liberan al motor transaccional de cargas de lectura.

4. Modelo de datos principal

Entidades con sus relaciones. En la entidad `Poliza`, la prima se *calcula* automáticamente en `@PrePersist / @PreUpdate` como `prima_total = canon_mensual × vigencia_meses`.

4.1 POLIZA (1 → N RIESGO)

Campo	Tipo / constraint	Descripción
id	BIGINT (PK, IDENTITY)	Identificador
numero_poliza	VARCHAR(50) UNIQUE	Número de póliza
tipo	ENUM 'INDIVIDUAL' 'COLECTIVA'	Tipo de póliza
estado	ENUM 'ACTIVA' 'RENOVADA' 'CANCELADA'	Estado del ciclo de vida
tomador / tomador_doc	VARCHAR(150)	Tomador (inmobiliaria en colectiva)
asegurado / beneficiario	VARCHAR(150)	Partes del contrato
vigencia_meses	INTEGER (> 0)	Vigencia (p. ej. 12)
fecha_inicio / fecha_fin	DATE	Vigencia contractual
canon_mensual	NUMERIC(14,2)	Canon mensual del arriendo
prima_total	NUMERIC(14,2)	Prima = canon × vigencia (calculada)
porcentaje_ipc_ultima_renovacion	NUMERIC(5,2)	Último IPC aplicado
fecha_ultima_renovacion	DATE	Fecha de última renovación

4.2 RIESGO (N → 1 POLIZA)

Campo	Tipo / constraint	Descripción
id / poliza_id	BIGINT (PK) / FK → polizas.id	Identificador y relación
direccion / ciudad	VARCHAR(200) / VARCHAR(100)	Inmueble asegurado
valor_canon	NUMERIC(14,2)	Canon del inmueble
descripcion_inmueble	TEXT	Descripción del bien
arrendatario_nombre / arrendatario_doc	VARCHAR(150)	Arrendatario
estado	ENUM 'ACTIVO' 'CANCELADO'	Estado del riesgo
fecha_creacion	DATE	Fecha de alta

4.3 AUDITORIA_CORE_LOG y RECORDATORIO (soporte)

Entidad	Campos clave	Propósito
AUDITORIA_CORE_LOG	poliza_id, evento, payload (JSONB), http_status, timestamp	Trazabilidad de envíos al CORE
RECORDATORIO	poliza_id, tipo (RENOVACION/IPC/CANON), fecha_limite, canal (EMAIL/SMS/PUSH), completado	Alertas de renovación y pago

5. Aspectos no funcionales (NFR)

5.1 Escalabilidad

- Kubernetes (EKS/GKE) con HPA basado en CPU y latencia HTTP.
- Caché distribuido Redis para pólizas vigentes e índices IPC.
- Read-replicas en PostgreSQL (lecturas del tablero separadas de las escrituras).

5.2 Logs y observabilidad (implementado en la prueba)

- OpenTelemetry con Correlation ID (TraceId) transversal hasta WebLogic (diseño).
- Micrometer + Prometheus + Grafana y centralización con Loki/EFK (diseño).
- **Implementado:** logs estructurados SLF4J en `CoreIntegrationService` y `CoreMockController` que evidencian cada intento de sincronización.

5.3 Tolerancia a fallos (implementado en la prueba)

- Resilience4j (Circuit Breaker, Retry con Exponential Backoff, RateLimiter) hacia WebLogic (diseño).
- Patrón Transactional Outbox + DLQ (diseño).
- **Implementado:** `notificarCambioCore()` envuelve la llamada en try/catch — si la capa WebLogic no responde, la transacción de negocio no se interrumpe y se registra la advertencia para reintento.

5.4 Versionamiento de APIs

- Versionado por URI (`/api/v1/polizas`) y Content Negotiation.
- Retrocompatibilidad: campos nuevos opcionales y headers `Deprecation` / `Sunset` .

Elaborado a partir de la implementación real: `spring-boot-backend/src`, `RESPUESTA_PRUEBA_TECNICA.md` y el diagrama Archify (`docs/seguros-polizas-architecture.html`). Diagramas generados con Archify.